Information Security

Lecture 5: Public-Key Cryptography, Integrity and Trust

A textbook treatment

Based on the lectures, TPs and quiz of Imane Fouad

October 25, 2026

Contents

How to read this book **2**

5 Public-Key Cryptography, Integrity and Trust **3**
5.1 Why Asymmetric 3
5.2 Number Theory for RSA 4
5.3 RSA 4
5.4 Hash Functions and Passwords 6
5.5 MACs and Digital Signatures 7
5.6 Certificates and PKI 9
5.7 Steganography 10
Chapter takeaways 11

How to read this book

Each concept in this book is presented in the same rhythm, and every section closes with a *Go deeper* box (the exact details, numbers and edge cases a demanding examiner asks about) followed by *Hard questions, answered.*

1. **Motivation and intuition.** Why do we care? What problem does the idea solve, and what picture should you hold in your head?
2. **Formal statement.** The precise definition or theorem, set in a coloured box so it is easy to find when revising.
3. **Proof.** Every claim is proved. Longer proofs are split into labelled steps, and the one idea that makes the proof work is isolated in a *Key insight* box.
4. **Interpretation and discussion.** What the result really says, where it applies, and which misreadings to avoid.
5. **Examples.** From "trivially small" to "genuinely useful."
6. **Techniques, classic problems and gotchas.** Computational shortcuts, proof patterns, exam favourites with one-line solutions, and the mistakes everyone makes once.

Margin notes carry the informal, "say it out loud" version of what is happening in the main text. Every major section opens with a *What are we trying to get?* box that states the goal in one breath before any mathematics, and closes with a summary box.

Prerequisites. Comfort with binary and hexadecimal, elementary probability, and the modular arithmetic of integers; everything else (finite fields for AES, the number theory behind RSA, Shannon entropy) is built where it is needed.

Chapter 5

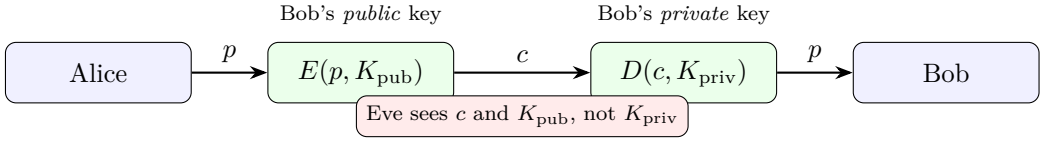
Public-Key Cryptography, Integrity and Trust

5.1 Why Asymmetric

What are we trying to get?

Symmetric cryptography needs a secret delivered in advance to every pair of parties. We want the idea that removes that requirement, the mathematical object it rests on, and the new problem it creates in exchange.

With one shared key per pair, N people need $\binom{N}{2} = N(N-1)/2$ keys (6 people: 15) and a secure channel for each. With key pairs, $2N$ keys (6 people: 12) and no pre-shared secret. The mailbox principle: anyone can drop a letter, only the owner can open the box. Diffie, Hellman and Merkle published the idea in 1976.



Definition 5.1.1 (One-way and trapdoor functions; requirements)**.** A **one-way function** $y = f(x)$ is easy to compute and infeasible to invert. A **trapdoor function** is one-way except for whoever holds secret information t, generated together with f by an algorithm Gen; the public key is f, the private key is t. A public-key system must make key generation, encryption with the public key and decryption with the private key *easy*, and deriving the private key from the public key or the message from the ciphertext *infeasible*. Candidate: integer factorisation. 6895601 is a product of two primes; finding them is slow, but given the trapdoor $t = 1931$ the division $6895601/1931 = 3571$ is immediate.

Key insight

Public keys remove the need for a *secret* channel and create the need for an *authentic* one: Alice must know that the public key is really Bob's, or a man in the middle substitutes his own (§5.5). Certificates (§5.6) exist for exactly this.

Hard questions, answered

Q. Why does asymmetric cryptography not eliminate the key-distribution problem but transform it?
A. Secrecy of the distributed key is no longer needed, authenticity is; a wrong public key gives Eve the plaintext.

Q. For $N = 100$ users, how many keys under each model?
A. Symmetric: $100 \cdot 99/2 = 4950$. Asymmetric: 200.

5.2 Number Theory for RSA

What are we trying to get?

RSA needs four facts about integers: congruences, inverses modulo n, Euler's totient, and Euler's theorem; plus fast exponentiation to compute with them.

Definition 5.2.1 (Modular arithmetic, inverses, totient)**.** $a \equiv b \pmod{n}$ iff $n \mid a - b$. An integer a has a multiplicative inverse modulo n iff $\gcd(a, n) = 1$; the extended Euclidean algorithm finds it (for tiny numbers, try $d = 1, 2, \ldots$ until $ed \bmod \varphi = 1$). **Euler's totient** $\varphi(n)$ counts the integers $1 \le a < n$ coprime to n; for distinct primes, $\varphi(pq) = (p-1)(q-1)$.

Theorem 5.2.2 (Euler; Fermat)**.** *If* $\gcd(a, n) = 1$ *then* $a^{\varphi(n)} \equiv 1 \pmod{n}$. *In particular (Fermat) for a prime* p *and* $p \nmid a$, $a^{p-1} \equiv 1 \pmod{p}$.

Proof. The map $x \mapsto ax$ permutes the $\varphi(n)$ residues coprime to n (it is injective because a is invertible). Multiplying all of them gives $\prod x \equiv a^{\varphi(n)} \prod x$, and $\prod x$ is invertible, so $a^{\varphi(n)} \equiv 1$. □

Technique: Fast exponentiation

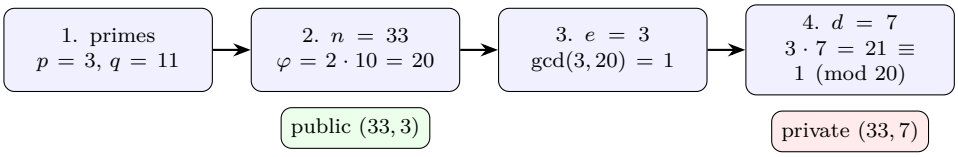
Square repeatedly and multiply the needed powers, reducing modulo n at every step: $26^7 \bmod 33$ via $26^2 \equiv 16$, $26^4 \equiv 16^2 = 256 \equiv 25$, $26^7 = 26^4 \cdot 26^2 \cdot 26 \equiv 25 \cdot 16 \cdot 26 \bmod 33 = 5$. Cost $O(\log e)$ multiplications instead of e.

5.3 RSA

What are we trying to get?

We want the algorithm, a proof that decryption undoes encryption, the worked numbers of the slides and the TP, and the three practical caveats (size of n, speed, determinism) that turn textbook RSA into usable RSA.

Definition 5.3.1 (RSA (Rivest, Shamir, Adleman, 1977))**.** Choose distinct large primes p, q (random, similar in size); $n = pq$; $\varphi = (p-1)(q-1)$; choose e with $1 < e < \varphi$ and $\gcd(e, \varphi) = 1$; compute d with $ed \equiv 1 \pmod{\varphi}$. Public key (n, e), private key (n, d); p, q, φ remain secret. Encrypt $c = m^e \bmod n$; decrypt $m = c^d \bmod n$.



Theorem 5.3.2 (Correctness of RSA)**.** *For every* $0 \leq m < n$, $(m^e)^d \equiv m \pmod{n}$.

Proof. $ed = 1 + k\varphi$ for some integer k. If $\gcd(m, n) = 1$, Theorem 5.2.2 gives $m^{ed} = m \cdot (m^{\varphi})^k \equiv m$. If $p \mid m$, both sides are 0 modulo p; modulo q (which does not divide m) Fermat gives $m^{ed} = m \cdot (m^{q-1})^{k(p-1)} \equiv m$; the congruence holds modulo p and q, hence modulo n by the Chinese remainder theorem. The case $q \mid m$ is symmetric. □

Worked example: Slides and the TP password

Slide, $m = 4$: $c = 4^3 = 64 \bmod 33 = 31$; $31^7 \bmod 33 = 4$. Exercise, $m = 5$: $c = 125 \bmod 33 = 26$; $26^7 \bmod 33 = 5$ (technique above). TP, $p = 5$, $q = 11$, $e = 3$, $C = 23$: $n = 55$, $\varphi = 40$, $d = 27$ ($81 = 2 \cdot 40 + 1$). $23^2 = 529 \equiv 34$; $23^4 \equiv 34^2 = 1156 \equiv 1$; so $23^{27} = (23^4)^6 \cdot 23^3 \equiv 23^3 = 23 \cdot 34 = 782 \equiv 12$. Password $m = 12$; check $12^3 = 1728 = 31 \cdot 55 + 23 = C$. A fresh pair: $p = 7$, $q = 13$, $e = 5$: $n = 91$, $\varphi = 72$, $d = 29$; $m = 2 \mapsto c = 32$.

Interpretation & discussion

Security rests on factoring n: with p and q one computes φ and then d. The RSA Factoring Challenge marks the frontier: RSA-768 (232 digits) factored December 2009, RSA-250 (829 bits) February 2020; 1024-bit keys "are out there", 2048 bits is the default. A 50-bit modulus (TP) falls to trial division up to $\sqrt{n} \approx 2^{25} \approx 3.4 \times 10^7$ in seconds; a 512-bit modulus has $\sqrt{n} \approx 2^{256}$ and needs the number field sieve (deprecated, but not trial division). RSA is slow and the message must be smaller than n, so it encrypts a *symmetric key*, not a file: the **hybrid** scheme draws a random AES key and IV, encrypts the message under AES-256-CBC, and encrypts the AES key under the recipient's RSA public key. Textbook RSA is deterministic and malleable: a 4-digit PIN is found by encrypting all 10^4 candidates and comparing; real systems use randomised padding (OAEP).

From the TP: OpenSSL mission 3

`openssl genrsa -out k.pem 512` writes an *unencrypted* private key: whoever reads the file owns it. `openssl rsa -in k.pem -text -noout` prints n, e, d, p, q; `-pubout` extracts the public key, which may be stored in plaintext; `-aes256` at generation encrypts the key file under a passphrase with AES-256-CBC. `pkeyutl -encrypt -pubin` works on a short file and fails on a large one. Factoring the 50-bit `otherSmall_pub.pem`: extract n with `-pubin -text`, trial-divide to $\sqrt{n}$, compute d; the same loop on a 512-bit modulus never finishes.

Go deeper: the details an examiner asks about

- e must be coprime to φ or d does not exist; the common choice is $e = 65537 = 2^{16} + 1$.
- "Similar in magnitude": a small prime is found by trial division; primes too close fall to Fermat's method $n = a^2 - b^2$; $p = q$ makes n a perfect square and φ wrong.
- The slide says "modulo a prime"; precisely, modulo $n = pq$.

Hard questions, answered

Q. Why combine AES and RSA for a long message?
A. RSA is slow and bounded by n; AES is fast and unbounded. RSA protects only the 32-byte key, AES does the bulk, and no pre-shared secret is needed.

Q. Eve knows (n, e) and the ciphertext of a 4-digit PIN. Attack?
A. Encrypt all 10^4 PINs under (n, e) and compare: textbook RSA is deterministic. Fix: OAEP padding.

5.4 Hash Functions and Passwords

What are we trying to get?

A hash compresses any input to a fixed fingerprint. We want the three resistance properties and how they relate, the proof that a hash alone gives no authenticity, and the password-storage story (dictionaries, salts, slow hashes).

Definition 5.4.1 (Cryptographic hash)**.** H maps an arbitrary-length input to a fixed-length digest; it is fast, lossy (the input cannot be reconstructed), and a tiny change in the input produces a very different digest. Requirements: **preimage resistance** (given y, hard to find any x with $H(x) = y$), **second-preimage resistance** (given x, hard to find $x' \neq x$ with $H(x') = H(x)$), **collision resistance** (hard to find *any* pair $m_1 \neq m_2$ with equal digests). Collisions exist because the output space is finite; the question is the work to find one. Generic attacks: preimages cost about 2^n for an n-bit digest, collisions about $2^{n/2}$ (birthday bound). Collision resistance implies second-preimage resistance; neither implies preimage resistance. MD5 and SHA-1 have practical collisions; SHA-2 (224/256/384/512) is current.

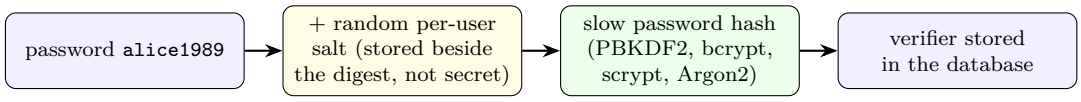
Theorem 5.4.2 (A hash alone gives no authenticity)**.** *If Alice sends $(m, H(m))$ over a channel Eve controls, Eve can replace it by $(m', H(m'))$ and Bob's check passes.*

Proof. H is public and takes no key; Eve computes $H(m')$ herself. Integrity against an adversary therefore needs a secret: a MAC key or a signing key (§5.5). A bare hash detects only *accidental* corruption. □

Theorem 5.4.3 (Birthday bound)**.** *Among k random n-bit digests the expected number of colliding pairs is $\approx k^2/2^{n+1}$, which reaches 1 at $k \approx 2^{n/2}$.*

Proof. There are $\binom{k}{2} \approx k^2/2$ pairs and each collides with probability 2^{-n}; by linearity of expectation the expected count is $k^2/2^{n+1}$. □

Password storage



Store H(password), never the password; at login hash the input and compare. A stolen digest is harder to use than plaintext, but *preimage resistance assumes an unknown high-entropy input*: a password drawn from a small dictionary is found by enumeration however good H is. **Dictionary attack**: precompute the digests of common passwords; unsalted, the same password gives the same digest for every user, so one table serves every account and equal digests reveal shared passwords. **Salt**: a random per-account value stored beside the digest and mixed into the hash; it need not be secret, because its purpose is to make each account's computation *different*: no shared table, one guessing campaign per user. Salts do not strengthen a weak password; *slow* password hashes raise the per-guess cost (OpenSSL's `-pbkdf2` is such a derivation).

Go deeper: the details an examiner asks about

- Hashing on the client before sending does not remove server-side hashing: the client-side hash *becomes* the password and can be replayed.
- "MD5 is broken" matters for signatures (an attacker crafting two documents with one digest, the 2008 rogue-CA case) more than for download checksums against accidental corruption, where a second preimage is still needed.

Hard questions, answered

Q. Two users share a password. Unsalted vs salted database?
A. Unsalted: identical digests; cracking one cracks both; precomputed tables apply to every account. Salted: different digests; separate work per account.

Q. Why is a 128-bit hash (MD5) considered too short for collisions even if the function were perfect?
A. By Theorem 5.4.3 a collision costs only 2^{64} work, within reach; preimages would still cost 2^{128}.

5.5 MACs and Digital Signatures

What are we trying to get?

Theorem 5.4.2 says integrity needs a secret. We want the two ways to add one, symmetric (MAC) and asymmetric (signature), what each guarantees and what each cannot, and why signatures are applied to a hash.

MAC (shared key K)
Alice: $t = \mathrm{MAC}(K, m)$
Bob: recompute, compare

Eve without K cannot forge t; but Bob *could* have: no non-repudiation

Signature (key pair)
Alice: $s = \mathrm{Sign}(\mathrm{priv}, H(m))$
Bob: $\mathrm{Verify}(\mathrm{pub}, H(m), s)$

only Alice holds priv: origin, integrity, non-repudiation

Definition 5.5.1 (MAC; digital signature)**.** A **message authentication code** uses a shared secret: Alice sends $(m, t = \mathrm{MAC}(K, m))$, Bob recomputes and compares; without K, Eve cannot produce a valid tag for a modified message. Constructions: the last block of CBC encryption of m (the sender needed key and message together; expensive), or hash-based schemes (hash the message and encrypt the digest symmetrically; hash the message concatenated with a secret value), which require a *cryptographic* hash. A MAC provides integrity and shared-key authentication, *no* confidentiality and *no* non-repudiation. A **digital signature** uses the sender's private key over the hash of the message and is verified with the sender's public key; it provides origin authentication, data integrity and signatory **non-repudiation** (NIST FIPS 186-4); algorithms RSA, DSA, ECDSA; no pre-shared key is needed.

Reading the formula

Why sign the hash. A 15 MB document under 1024-bit RSA would require thousands of slow operations, and signing raw messages of the attacker's choice exposes the private key to algebraic attacks; the digest is short and fixed-size, so the signature's size and cost are independent of the message. The scheme then needs collision resistance: if Eve finds m_1, m_2 with one digest and gets m_1 signed, the signature validates m_2. *Why no non-repudiation from a MAC.* Both parties hold K, so Bob could have produced the tag himself; a judge cannot tell who did. A signature uses a key only Alice holds.

	Confidentiality	Integrity	Authentication	Non-repudiation
Encryption	yes	no	no	no
MAC	no	yes	yes (shared key)	no
Signature	no	yes	yes	yes

Confidentiality and authenticity are separate: secrets sent anonymously (C without A), public information from a trusted source (A without C), secure communication (both). None of the three gives availability.

Hard questions, answered

Q. A confidential contract that Bob cannot later deny having received from Alice. Primitives and order?
A. Sign with Alice's private key over $H(m)$ (authenticity, non-repudiation), then encrypt for Bob (hybrid). Sign first so the signature binds the plaintext.

Q. Is $\mathrm{MAC} = H(K \| m)$ safe?
A. Not with Merkle–Damgård hashes (MD5, SHA-1, SHA-256): length extension lets Eve compute $H(K \| m \| \mathrm{pad} \| m')$ without K. HMAC (or SHA-3) fixes it.

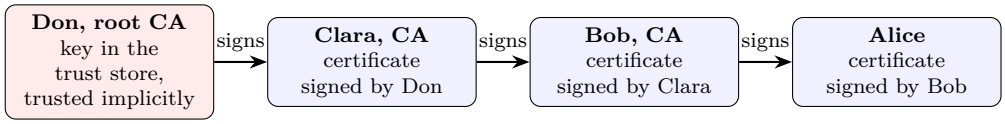
Q. Why can a MAC not serve as evidence in court?
A. The verifier holds the same key as the signer and could have produced the tag; a third party cannot attribute it. Only a signature attributes.

5.6 Certificates and PKI

What are we trying to get?

Signatures need the verifier to hold the signer's authentic public key, which is the problem that asymmetric cryptography created in §5.1. We want the recursion that solves it, where it bottoms out, and what depends on that base case.

To verify Alice's signature we need her public key; she cannot prove remotely that a key is hers by signing a message with that same key (verifying it would need the key in question). So someone else signs a message binding Alice's identity to her key; that signer's key is vouched for by another; and so on, until a key that "was shipped to us just for this" and is trusted implicitly.



to trust Alice's key, verify each signature from right to left until a root in the trust store

Definition 5.6.1 (Certificate, CA, chain of trust, CSR, PKI)**.** A **certificate** is a message containing an identity and a public key, signed by someone else; each signer is a **certificate authority** (CA); the implicitly trusted one is a **root CA**; the signatures form a **chain of trust**. Obtaining a certificate: (1) generate a key pair and keep the private key secret forever; (2) present the public key and identity to the CA in a **certificate signing request** (everything in the certificate except the signature) with proof of identity; (3) the CA verifies and signs. Format: X.509 (RFC 5280). **PKI** (RFC 4949): the set of hardware, software, people, policies and procedures needed to create, manage, store, distribute and *revoke* certificates; it includes a **trust store**, the list of root CAs and their keys, chosen by the OS or application vendor.

Interpretation & discussion

The base case is trust by *distribution*, not by cryptography: root keys arrive with the operating system. Whoever controls the trust store controls everything above it. Organisations add a private root to sign internal certificates; malware that adds a root can have it sign a certificate for *any* site and mount man-in-the-middle attacks on every HTTPS connection from that machine; some "security" appliances do exactly this to inspect encrypted traffic, which the course calls stupid and dangerous, since one compromised CA key breaks confidentiality for everyone behind it. What a browser checks: the server certificate's signature by the intermediate, the intermediate's by a root in the store, validity dates, that the name matches, revocation status; only then does it use the server's public key for key exchange.

Hard questions, answered

Q. Why can't Alice self-sign to prove her key is hers?
A. Verifying would need her public key, the thing in question; a self-signature shows only that the key pair's holder signed. Trust must enter from outside.

Q. CSR versus the CA generating the key pair: what changes?
A. With a CSR the private key never leaves Alice; the CA attests only the public key. If the CA generates the pair it holds the private key, which destroys non-repudiation and confidentiality guarantees.

Q. Malware adds a root CA to one laptop. What can the attacker do, and what stays safe?
A. Issue a certificate for any site that laptop's browser will accept, hence MITM every HTTPS connection from it (read and alter). Other machines are unaffected; certificate pinning or fingerprint checks may detect it.

5.7 Steganography

Cryptography hides the *meaning* of a message; steganography hides its *existence*. Simmons (1983) brought it into computer science as the *Prisoners' Problem*: Alice and Bob communicate through a warden; the carriers are *stego objects*, the hidden data the *payload*; a *passive* warden only observes (a detection problem for the warden), an *active* warden alters the objects (a robustness problem for Alice). The first recorded technique is from Greece around 440 BC. Null ciphers hide a message in the n-th letters of words (third letters of "Fishing freshwater bends and saltwater coasts rewards anyone feeling stressed…" spell "Send Lawyers, Guns, and Money"; first letters of the first paragraph and second letters of the second of the WWI telegram spell "PERSHING SAILS FROM N.Y. JUNE 1"). Types: text, image, audio, video, network and email steganography; the size of the Internet is what makes hiding effective. **LSB steganography** overwrites the least significant bit of each pixel or sample with payload bits: each value changes by at most 1/255, invisible; capacity one bit per byte; detectable by LSB statistics; destroyed by recompression. Encrypt the payload first so that detection does not reveal meaning.

Hard questions, answered

Q. Hide 'A' (`01000001`) in pixels `10110010, 10110011, 10110010, 10110011, 10110010, 10110011, 10110010, 10110011`.
A. Replace each LSB: `10110010, 10110011, 10110010, 10110010, 10110010, 10110010, 10110010, 10110011`; each value moves by at most one.

Q. Which warden defeats LSB trivially, and how?
A. The active warden: re-encode the image (JPEG, blur, resample); the LSB plane is destroyed while the picture survives for the receiver.

Chapter takeaways

Summary: Public-key cryptography, integrity, trust

- $N(N-1)/2$ vs $2N$ keys; trapdoor one-way functions; the authenticity of public keys becomes the problem.
- RSA: $n = pq$, φ, $ed \equiv 1$; correct by Euler (Theorem 5.3.2); secure by factoring; hybrid with AES; padding against determinism.
- Hash: preimage, second preimage, collision ($2^{n/2}$, Theorem 5.4.3); keyless, so no authenticity (Theorem 5.4.2); passwords: salt against shared tables, slow hashes against speed.
- MAC (shared key: integrity and authentication) vs signature (private key over $H(m)$: plus non-repudiation); encryption gives confidentiality only.
- Certificates bind identity to key by a CA's signature; chains end at a distributed root; the trust store is the root of everything; PKI includes revocation.
- Steganography hides existence; LSB; passive vs active warden.